

Lab_03_Thomas_Travassos

Thomas Toledo Travassos

###Lab_03

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
```

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

-Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;

-Para o arquivo de vôos:

a.Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`; b.Leia apenas 1 milhão de linhas por vez; c.Remova vôos em que o aeroporto de destino comece com a letra 1; d.Remova registros em que existam pelo menos uma coluna faltante; e.Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; f.Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
airports <- read_csv(
  "datasets/airports.csv",
  col_select = c(
    IATA_CODE,
    CITY,
    STATE,
    LATITUDE,
    LONGITUDE
  )
)
```

Rows: 322 Columns: 5

-- Column specification -----

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
airlines <- read_csv(
  "datasets/airlines.csv"
)
```

Rows: 14 Columns: 2

-- Column specification -----

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
getStats <- function(input, pos) {
  input %>%
    filter(substr(DESTINATION_AIRPORT, 1, 1) != "I") %>%
    filter(!is.na(DESTINATION_AIRPORT),
           !is.na(ARRIVAL_DELAY)) %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      total = n(),

```

```

      soma = sum(ARRIVAL_DELAY),
      .groups = "drop"
    )
  }
}

```

```

stats_lotes <- read_csv_chunked(
  "datasets/flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

```

computeStats <- function(stats) {

  stats %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      total = sum(total),
      soma = sum(soma),
      .groups = "drop"
    ) %>%
    mutate(
      atraso_medio = soma / total
    )

}

```

```

flights <- computeStats(stats_lotes)

```

```

head(flights)

```

```

# A tibble: 6 x 4
  DESTINATION_AIRPORT total  soma atraso_medio
  <chr>              <int> <dbl>         <dbl>
1 10135                235   278           1.18
2 10136                194   237           1.22
3 10140               1811  2954           1.63

```

4	10141	71	-134	-1.89
5	10146	87	620	7.13
6	10154	27	189	7

2. Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela `flights`; b. Identifique a coluna que é a chave na tabela `airports`; c. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`? d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves; e. Armazene a tabela resultante no objeto `flights`.

```
anti_join(
  flights,
  airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
```

```
# A tibble: 307 x 4
  DESTINATION_AIRPORT total  soma atraso_medio
  <chr>               <int> <dbl>         <dbl>
1 10135                235   278           1.18
2 10136                194   237           1.22
3 10140               1811  2954           1.63
4 10141                 71  -134          -1.89
5 10146                 87   620           7.13
6 10154                 27   189            7
7 10155                141 -285          -2.02
8 10157                118  1110           9.41
9 10158                246   880           3.58
10 10165                 9  -142          -15.8
# i 297 more rows
```

```
flights <- left_join(
  flights,
  airports %>% select(IATA_CODE, CITY, STATE),
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
```

```
head(flights)
```

```
# A tibble: 6 x 6
  DESTINATION_AIRPORT total  soma atraso_medio CITY  STATE
  <chr>                <int> <dbl>         <dbl> <chr> <chr>
1 10135                 235   278         1.18 <NA> <NA>
2 10136                 194   237         1.22 <NA> <NA>
3 10140                1811  2954         1.63 <NA> <NA>
4 10141                 71  -134        -1.89 <NA> <NA>
5 10146                 87   620         7.13 <NA> <NA>
6 10154                 27   189          7    <NA> <NA>
```

3.Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports %>%
  group_by(STATE) %>%
  summarise(
    aeroportos = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(aeroportos))
```

```
# A tibble: 54 x 2
  STATE aeroportos
  <chr>         <int>
1 TX             24
2 CA             22
3 AK             19
4 FL             17
5 MI             15
6 NY             14
7 CO             10
8 MN              8
9 MT              8
10 NC             8
# i 44 more rows
```

4.Apresente um mapa representando todos os atrasos observados por aeroporto.

a.Carregue o pacote `leaflet`; b.Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico; c.Armazene o grafico em b) numa variável chamada `this`; d.Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```

library(leaflet)

flights <- left_join(
  flights,
  airports %>% select(IATA_CODE, LATITUDE, LONGITUDE),
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)

this <- leaflet(flights) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste(
      DESTINATION_AIRPORT,
      "<br>Cidade:", CITY,
      "<br>Estado:", STATE,
      "<br>Atraso médio:", round(atraso_medio, 2)
    )
  )

```

Warning in validateCoords(lng, lat, funcName): Data contains 310 rows with either missing or invalid lat/lon values and will be ignored

```

Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()

```

```
[1] "2026-08-30 11:56:06 -03"
```